UD2.EXAME Práctico

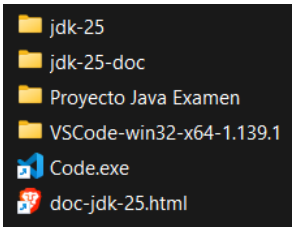
DAM1-Programación 2026-27
2026/11/09

“Torneo 3x3" — JDBC, DAO, DTO y Transacciones sobre SQLite **3**
Esquema BD 3
Arquitectura del proyecto (ya implementada) 4
Convenciones del proyecto 4
Ejercicios **5**
1. EquipoDAOImpl.listarEquipos (0,5 ptos) 5
2. EquipoDAOImpl.insertarEquipo (1,5 ptos) 5
3. PartidoDAOImpl.buscarPartido (1,5 ptos) 5
4. JugadorDAOImpl.listarJugadoresDeEquipo (0,5 ptos) 5
5. JugadorDAOImpl.buscarJugador (1 pto) 5
6. JugadorDAOImpl.actualizarAlturaJugador (1 pto) 6
7. PartidoDAOImpl.rankingAnotadores (2 ptos) 6
8. EstadisticaDAOImpl.insertar (0,5 ptos) 6
9. TorneoService.cancelarPartido (2,5 ptos) 6
Orientación de Evaluación **8**
Datos de prueba **9**
Incluidos en la BD 9
Pruebas sugeridas con el menú del Main 9
Anexo. Cláusula “record” en DTOs **10**
Anexo. Sobre TorneoService **11**

- **Apaga y guarda tu teléfono móvil. Coloca tu mochila y abrigo junto a la entrada para recogerlos al terminar y salir.**
- Realiza los programas **individualmente y en silencio**. En caso contrario se retirará el examen y se calificará con un cero.
- Si necesitas **consultar al profesor** llama su atención levantando la mano y espera a que se acerque.
- **Cuida el código fuente** y realiza programas bien estructurados, legibles, con comentarios, líneas en blanco, identificadores adecuados, etc.
- **Cuida la interacción con el usuario**, presentando la información de forma clara y ordenada, sin faltas de ortografía.
- **Utiliza los casos de prueba** de ejemplo si los hay para probar tu programa y añade tus propios casos.
- NO se valorarán programas con **errores de compilación**.
- NO se permite el uso de sentencias, estructuras o **técnicas no vistas en clase**. El profesor podrá solicitar **explicaciones adicionales** sobre el código entregado.
- Utiliza los **últimos minutos** de examen para verificar que tu entrega es correcta.

Entorno de Programación proporcionado:

- IDE Visual Studio Code con Java Extensions Pack
- JDK 25 con documentación de la API.
- Proyecto Java para realizar el examen.
- Librerías y dependencias necesarias.
- Apuntes y material didáctico adicional en PDF.



Entrega Proyecto completo comprimido con los TODO resueltos.

Tiempo estimado: 2 horas

“Torneo 3x3" — JDBC, DAO, DTO y Transacciones sobre SQLite

Se te entrega el proyecto torneo3x3 con la aplicación parcialmente implementada. La base de datos torneo3x3.db (SQLite, ya poblada) modela un campeonato de baloncesto 3x3 con equipos, jugadores, partidos y estadísticas.

Tu tarea es completar los métodos marcados con TODO en las clases indicadas. El resto del proyecto (DTOs, interfaces DAO, ConexionDB, TorneoService.registrarResultado, Main) está ya implementado y no debes modificarlo salvo donde se indique.

Cada método TODO lanza **UnsupportedOperationException** hasta que lo implementes. Puedes usar el Main (menú de consola) para probar cada tarea de forma aislada.

Esquema BD

```
CREATE TABLE equipos (
    id_equipo     INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre        TEXT NOT NULL UNIQUE
);

CREATE TABLE jugadores (
    id_jugador    INTEGER PRIMARY KEY AUTOINCREMENT,
    id_equipo     INTEGER NOT NULL REFERENCES equipos(id_equipo),
    dorsal        INTEGER NOT NULL,
    nombre        TEXT NOT NULL,
    altura_cm     INTEGER,
    UNIQUE (id_equipo, dorsal)
);

CREATE TABLE partidos (
    id_partido    INTEGER PRIMARY KEY AUTOINCREMENT,
    id_local      INTEGER NOT NULL REFERENCES equipos(id_equipo),
    id_visitante  INTEGER NOT NULL REFERENCES equipos(id_equipo),
    fecha         TEXT NOT NULL,          -- 'YYYY-MM-DD'
    puntos_local  INTEGER,
    puntos_visitante INTEGER,
    estado        TEXT NOT NULL DEFAULT 'PENDIENTE'
                  CHECK (estado IN ('PENDIENTE','JUGADO','CANCELADO'))
);

CREATE TABLE estadisticas (
    id_estadistica INTEGER PRIMARY KEY AUTOINCREMENT,
    id_partido     INTEGER NOT NULL REFERENCES partidos(id_partido),
```

```
    id_jugador     INTEGER NOT NULL REFERENCES jugadores(id_jugador),
    puntos         INTEGER NOT NULL DEFAULT 0,
    UNIQUE (id_partido, id_jugador)
);
```

Fichero SQLite
La BD contiene equipos, jugadores, y partidos (algunos PENDIENTES) y estadísticas de los partidos JUGADOS.

Arquitectura del proyecto (ya implementada)

```
src/main/java/examen/xxx/torneo3x3/
├── conexion/ConexionDB.java              ← COMPLETO (fábrica de
conexiones)
├── excepciones/AccesoDatosException.java ← COMPLETO
├── dto/                                  ← COMPLETOS (records)
│   ├── EquipoDTO.java
│   ├── JugadorDTO.java
│   ├── PartidoDTO.java
│   ├── EstadisticaDTO.java
│   └── RankingDTO.java
├── dao/                                  ← INTERFACES COMPLETAS
│   ├── EquipoDAO.java
│   ├── JugadorDAO.java
│   ├── PartidoDAO.java
│   └── EstadisticaDAO.java
├── dao/impl/                             ← AQUÍ ESTÁ TU TRABAJO
│   ├── EquipoDAOImpl.java      ← TODO (ejercicios 1 y 2)
│   ├── JugadorDAOImpl.java     ← TODO (ejercicios 4, 5, 6)
│   ├── PartidoDAOImpl.java     ← TODO (ejercicios 3 y 7)
│   └── EstadisticaDAOImpl.java ← TODO (ejercicio 8)
├── servicio/
│   └── TorneoService.java      ← registrarResultado COMPLETO (referencia)
│                                 cancelarPartido TODO (ejercicio 9)
└── Main.java                   ← COMPLETO (menú de pruebas)
```

Convenciones del proyecto

1. Los DAOs reciben la Connection por parámetro. No la abren ni la cierran. Quien la abre, la cierra.
2. ConexionDB.getConexion() devuelve siempre una conexión nueva y activa
3. Uso exclusivo de PreparedStatement en consultas con parámetros. Penalización si se usa Statement con concatenación.

4. try-with-resources obligatorio para PreparedStatement y ResultSet.
5. Envolver SQLException en AccesoDatosException cuando corresponda (los métodos declaran throws SQLException; el Main ya captura ambas).
6. No se permite devolver ResultSet fuera de la capa DAO.
7. Los DTOs implementados como record y no se modifican.

Ejercicios

1. EquipoDAOImpl.listarEquipos (0,5 ptos)

```
List<EquipoDTO> listarEquipos(Connection conn) throws SQLException
```

Devuelve todos los equipos ordenados por nombre. Usa PreparedStatement, ResultSet y try-with-resources.

2. EquipoDAOImpl.insertarEquipo (1,5 ptos)

```
int insertarEquipo(EquipoDTO equipo, Connection conn) throws SQLException
```

Inserta un equipo y devuelve el id_equipo autogenerado usando getGeneratedKeys().

Si el nombre ya existe, SQLite lanza una SQLException con mensaje "UNIQUE constraint failed: equipos.nombre": captúrala y lanza AccesoDatosException("Equipo duplicado: " + nombre). El resto de SQLException se propagan tal cual.

3. PartidoDAOImpl.buscarPartido (1,5 ptos)

```
PartidoDTO buscarPartido(int id, Connection conn) throws SQLException
```

Devuelve el partido con sus datos y los nombres de los equipos local y visitante. Devuelve null si no existe.

Ten en cuenta que puntos_local y puntos_visitante pueden ser NULL (partidos PENDIENTE). Usa rs.getObject(...) para leerlos como Integer.

4. JugadorDAOImpl.listarJugadoresDeEquipo (0,5 ptos)

```
List<JugadorDTO> listarJugadoresDeEquipo(int idEquipo, Connection conn) throws SQLException
```

Devuelve los jugadores de un equipo ordenados por dorsal. altura_cm puede ser NULL.

5. JugadorDAOImpl.buscarJugador (1 pto)

```
JugadorDTO buscarJugador(int id, Connection conn) throws SQLException
```

Devuelve el jugador por id como Optional. Mismo patrón que el ejercicio 3.

Nota: este método lo usa TorneoService.registrarResultado. Si no lo implementas, el método de referencia del servicio fallará al ejecutarse.

6. JugadorDAOImpl.actualizarAlturaJugador (1 pto)

```
int actualizarAlturaJugador(int id, int altura, Connection conn) throws SQLException
```

Valida que 150 <= altura <= 230; si no, lanza IllegalArgumentException("Altura fuera de rango [150,230]: " + altura) antes de tocar la BD.
Después ejecuta el UPDATE y devuelve las filas afectadas.

7. PartidoDAOImpl.rankingAnotadores (2 ptos)

```
List<RankingDTO> rankingAnotadores(Connection conn) throws SQLException
```

Devuelve, por jugador: nombre del jugador, nombre del equipo y total de puntos sumados solo en partidos JUGADO, ordenado descendentemente por total.

Sugerencia: Utiliza una consulta JOIN a estadisticas, jugadores, equipos y partidos, GROUP BY y SUM.

8. EstadisticaDAOImpl.insertar (0,5 ptos)

```
void insertar(EstadisticaDTO e, Connection conn) throws
SQLException
```

Inserta una fila en estadisticas. Lo usa TorneoService.registrarResultado.

9. TorneoService.cancelarPartido (2,5 ptos)

```
void cancelarPartido(int idPartido)
```

Implementa este método siguiendo exactamente el mismo patrón de transacción que TorneoService.registrarResultado.

Reglas de negocio:

1. El partido debe existir.
2. El partido debe estar en estado PENDIENTE; si no, error.
3. Al cancelar, se borran sus estadísticas (por si acaso hubiera).
4. El estado final del partido debe ser CANCELADO.
5. Todo en una única transacción: si algo falla, rollback y lanzar AccesoDatosException.
6. En finally, restaurar setAutoCommit(true) y cerrar la conexión.

Métodos de DAO necesarios (ya están en las interfaces):

1. PartidoDTO partidoDAO.buscarPartido(int id, Connection conn);
2. int partidoDAO.actualizarEstado(int id, String estado, Connection conn);
3. int estadisticaDAO.borrarPorPartido(int idPartido, Connection conn);

Sugerencia: copia la estructura de registrarResultado y adapta las validaciones y las operaciones.

Estructura:
1. Validaciones de negocio SIN BD (puntos negativos, duplicados)
2. conn = ConexionDB.getConexion()
3. conn.setAutoCommit(false)
4. Validaciones contra BD + operaciones con DAOs (misma conn)
5. conn.commit()
6. catch → conn.rollback() + relanzar AccesoDatosException
7. finally → conn.setAutoCommit(true) + conn.close()

No inventes otra forma de gestionar la transacción. No cambies este patrón. Si tu cancelarPartido no sigue estos siete pasos, revísalo.

Orientación de Evaluación

- Cada ejercicio puntúa sobre la nota numérica indicada entre paréntesis.
- A modo de orientación se valorará:
    - Uso correcto de PreparedStatement y try-with-resources
    - Corrección de las consultas SQL (JOIN, GROUP BY, ORDER BY, getGeneratedKeys)
    - Validaciones y manejo de excepciones
    - Cierre adecuado de recursos
    - **Buenas prácticas de programación:** elección de tipos e identificadores, legibilidad, reglas de estilo Java, mensajes claros en la interacción con el usuario, etc.
    - **Funcionalidad:** que el programa haga lo que se pide con exactitud.
    - Se penalizará modificar DTOs, interfaces o Main sin justificación.

Datos de prueba

Incluidos en la BD

- Partido 21 (PENDIENTE, Titanes vs Cementerio): listo para registrarResultado (referencia).
- Partido 22 (PENDIENTE, Tormenta BC vs Novatos BC): listo para cancelarPartido (ejercicio 9).
- Equipo "Novatos BC" (id 2): sin jugadores; útil para probar eliminarEquipo (referencia).
- Equipo "Titanes 3x3" (id 1): con 5 jugadores y 4 partidos jugados; eliminarEquipo(1) debe devolver false.
- Equipo "Tormenta BC" (id 9): con 1 partido, sin jugadores; eliminarEquipo(9) debe devolver false.

Pruebas sugeridas con el menú del Main

Opción	Ejercicio	Prueba sugerida
1	1	Debe listar 9 equipos ordenados
2	2	Insertar "Prueba BC" → id nuevo; repetir → AccesoDatosException
3	3	Buscar partido 21 → datos completos con nombres; buscar 999 → null
4	4	Equipo 1 → 5 jugadores ordenados por dorsal
5	5	Jugador 1 → datos; jugador 9999 → vacío
6	6	Altura 185 → 1 fila; altura 300 → IllegalArgumentException
7	7	Ranking ordenado por puntos descendente
8	8	Insertar estadística para partido pendiente 21 y jugador 1
9	9	Cancelar partido 22 → OK; cancelar 21 (ya JUGADO) → error + rollback

Anexo. Cláusula “record” en DTOs

En Java, record es un tipo especial de clase introducido en Java 16 que permite crear clases destinadas principalmente a almacenar datos, evitando tener que escribir mucho código repetitivo.

Ejemplo:

```java
public record EquipoDTO(int idEquipo, String nombre) {
    @Override
    public String toString() {
        return "[" + idEquipo + "] " + nombre;
    }
}
```

El código anterior declara un DTO (Data Transfer Object) llamado `EquipoDTO` que contiene dos atributos:

- `idEquipo`, de tipo `int`.
- `nombre`, de tipo `String`.

El compilador genera automáticamente:

- Un atributo privado y final por cada componente del record.
- Un constructor con todos los parámetros.
- Un método de acceso para cada atributo.
- `equals()`.
- `hashCode()`.
- `toString()`.

¿Cómo se utiliza?

Crear un objeto:

```java
EquipoDTO equipo = new EquipoDTO(1, "Breogán");
```

Acceder a sus atributos:

```java
System.out.println(equipo.idEquipo());
System.out.println(equipo.nombre());
```

Observa que los métodos de acceso se llaman `idEquipo()` y `nombre()`, no llevan el prefijo `get` como en los JavaBeans tradicionales.

Importante! Los componentes de un `record` **son inmutables**: sus referencias o valores no se pueden reasignar después de construir el objeto. **No tienen setters**.

Los `record` son especialmente interesantes para los DTO porque normalmente estos objetos solo sirven para transportar información entre capas de la aplicación.

Anexo. Sobre TorneoService

Se incluye una clase TorneoService que hace de intermediario entre los DAO y el Main.

- El servicio responde a "¿qué operación de negocio estoy haciendo?", el DAO a "¿qué SQL ejecuto?".
- Un servicio = un caso de uso completo. registrarResultado = comprobar + actualizar partido + insertar N estadísticas, todo o nada.
- El servicio no escribe SQL. Llama a DAOs.
- El servicio no conoce la presentación. No imprime menús ni lee de teclado. Devuelve o lanza excepciones.
- El servicio es el único que gestiona transacciones. Los DAOs no saben si están dentro de una transacción o no; solo usan la Connection que reciben. Eso es lo que permite componer varios DAOs en una sola transacción.

Un servicio solo debe existir cuando hay:

- Varias operaciones de DAO que deben ser atómicas (transacción).
- Reglas de negocio que no son SQL (validaciones, cálculos, coordinación).
- Un caso de uso que el cliente quiere invocar como una sola operación.
- Para un SELECT suelto como "listar equipos", no hace falta servicio. El Main puede usar el DAO directamente.